Competitive Programming

Section 50: Advanced Algorithms Mixed Contest

Competitive Programming Course

2026-10-24

Table of contents

1 Mixed advanced contest habits 3

2 Nineteen original problems 4

3 Diagnose the mathematical object 5

4 Proof checkpoints 6

5 Implementation and testing 7

6 Exercises 8

1 Mixed advanced contest habits

At this level, a powerful tool is useful only after the mathematical model has earned it. This contest tests whether you can choose that model without being told the technique in the statement.

2 Nineteen original problems

The contest contains nineteen self-contained packages, A–S. The seven new problems M–S bring each technical topic in sections 43–49 to two main problems across the section-50 set. Together with section 42's coverage of section 41, this closes the gap between the two mixed contests.

Problem	Title
A	DAG Grundy
B	General CRT
C	Fast Fibonacci
D	Fibonacci Range Sum
E	Pattern Prefix Matches
F	Suffix LCP Queries
G	Domino Placement
H	Weighted Tree Distances
I	Balanced Subset Difference
J	Monge Partition Table
K	Online Line DP
L	Profitable Disjoint Routes
M	Dial Alignment
N	Archive Reassembly
O	Relay Throughput
P	Last Alarm
Q	Exact Beacon Codes
R	Warehouse Blocks
S	Parabolic Signals

Each package contains its statement, public student stubs, both reference languages, and tests. All seven new packages include explicit input constraints, independent small oracles, and maximum-size workloads. The checker runs A–S.

For three shorter rounds, attempt A–F, G–L, then M–S. M–S are grouped by source topic rather than difficulty. L is an optional extension requiring its editorial's extra material; leave it aside until the main set is complete. Use the coverage map in the README only after choosing your own models.

3 Diagnose the mathematical object

Before coding, write down what a candidate solution represents. Identify the allowed choices, the quantity being optimized or counted, and the precise meaning of one state or maintained object.

```
What makes a candidate valid?
Which choices interact, and which can be separated?
What changes when I move one endpoint, root, or query position?
What do the largest inputs rule out?
```

Test your proposed model on a tiny case before optimizing it. A repeated value can expose accidental deduplication; a boundary endpoint can expose an indexing error; an impossible instance can expose an invalid default answer.

4 Proof checkpoints

A reduction needs a mapping in both directions, with the objective preserved. A recurrence needs an exhaustive description of its last choice. A greedy choice needs an exchange or dominance argument. An optimization needs its ordering assumptions proved for the actual cost function, not borrowed from a similar-looking exercise.

Write those obligations before opening the corresponding source editorial. The mixed lesson intentionally leaves the individual reductions to your attempt.

5 Implementation and testing

Keep public `solve.*` files as stubs and put personal solutions in the private submissions overlay. The reference checker can be run separately:

```
CP_TARGET=solution python3 sections/50_advanced_algorithms_mixed_contest/check.py --keep-goi
```

Use independent exhaustive oracles on small inputs. For example, enumerating all legal choices can check a fast implementation without sharing its internal state or transition formula. Agreement between two optimized implementations alone does not establish correctness.

Performance tests serve a separate purpose. A small random case rarely rejects a quadratic method. M–S each include two full-limit workloads with expected answers derived from their construction. They run under the normal time limit. Use wide integers for totals and products, and remember that a valid optimum can be negative or can count many subsets.

6 Exercises

Attempt all eighteen main problems and the optional extension L. For each, record the model, proof obligation, target complexity, and one counterexample to a tempting wrong approach. Then use the full editorial to upsolve and rerun the judge after correcting your implementation.